

PROJET FIL ROUGE · CDA 2026

Amoura

Application web de restaurant · Réservation & fidélité

JALON 3 – MARS 2026

Modélisation de la base de données

Formation CDA · SGBD cible : MariaDB · ORM : Doctrine

Amoura

1. Démarche de modélisation

Pour concevoir la base de données d'Amoura, j'ai suivi la méthode MERISE en trois niveaux successifs :

- **MCD** — modélisation métier des entités et de leurs relations, indépendamment de toute technologie.
- **MLD** — traduction en modèle relationnel : tables, clés primaires et étrangères.
- **MPD** — implémentation concrète pour MariaDB avec les types SQL, contraintes et index.

L'application couvre deux domaines fonctionnels : la **gestion des réservations** (clients, tables, créneaux) et le **programme de fidélité** (points, récompenses, historique). La base compte au total **8 tables**.

2. Dictionnaire des données

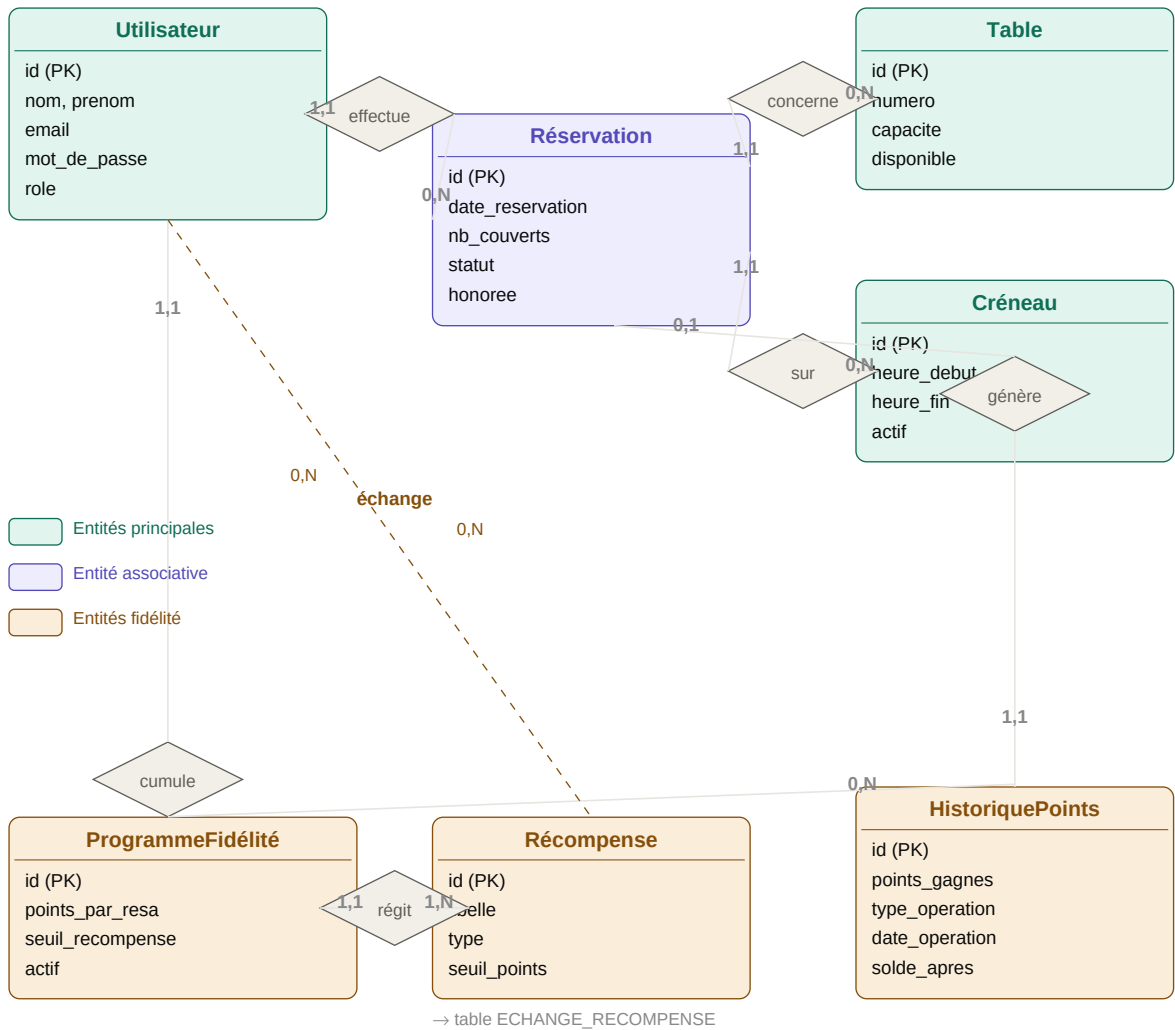
Ce dictionnaire liste toutes les entités identifiées, leurs attributs et les contraintes associées.

Attribut	Type	Contrainte	Signification
UTILISATEUR			
id	Entier auto	PK	Identifiant unique
nom, prenom	Chaîne 50	NOT NULL	Identité du client
email	Chaîne 180	UNIQUE, NOT NULL	Email de connexion
mot_de_passe	Chaîne 255	NOT NULL	Hash bcrypt/Argon2
role	Enum	DEFAULT client	client ou admin
created_at	DateHeure	NOT NULL	Date d'inscription
TABLE			
id	Entier auto	PK	Identifiant
numero	Entier	UNIQUE	Numéro de table (1-15)
capacite	Entier	CHECK 1-20	Nombre de places
disponible	Booléen	DEFAULT true	Table active ou désactivée
CRENEAU			
id	Entier auto	PK	Identifiant
heure_debut	Heure	NOT NULL	Ex : 12:00
heure_fin	Heure	NOT NULL	Ex : 14:30
actif	Booléen	DEFAULT true	Créneau ouvert ou fermé
RESERVATION			
id	Entier auto	PK	Identifiant
date_reservation	Date	NOT NULL	Jour de la réservation
nb_couverts	Entier	CHECK 1-20	Nombre de personnes
statut	Enum	NOT NULL	en_attente / confirmee / annulee
honoree	Booléen	DEFAULT false	Réservation effectivement honorée
utilisateur_id	Entier	FK → UTILISATEUR	Client ayant réservé
table_id	Entier	FK → TABLE	Table attribuée
creneau_id	Entier	FK → CRENEAU	Créneau horaire
PROGRAMME_FIDELITE			

id	Entier auto	PK	Identifiant (1 seule ligne)
points_par_resa	Entier	DEFAULT 10	Points par réservation honorée
seuil_recompense	Entier	DEFAULT 100	Points pour débloquent une récompense
actif	Booléen	DEFAULT true	Programme activé ou non
RECOMPENSE			
id	Entier auto	PK	Identifiant
libelle	Chaîne 100	NOT NULL	Ex : Dessert offert
type	Enum	NOT NULL	reduction / boisson / dessert / autre
seuil_points	Entier	NOT NULL	Points nécessaires
programme_id	Entier	FK → PROGRAMME_FIDELITE	Programme associé
HISTORIQUE_POINTS			
id	Entier auto	PK	Identifiant
points_gagnes	Entier	NOT NULL	Positif (gain) ou négatif (échange)
type_operation	Enum	NOT NULL	gain_reservation / echange_recompense
date_operation	DateHeure	NOT NULL	Horodatage
solde_apres	Entier	NOT NULL	Solde après opération
utilisateur_id	Entier	FK → UTILISATEUR	Client concerné
reservation_id	Entier	FK → RESERVATION (nullable)	Resa liée si gain
ECHANGE_RECOMPENSE			
id	Entier auto	PK	Identifiant
date_echange	DateHeure	NOT NULL	Date de l'échange
statut	Enum	DEFAULT utilise	utilise ou annule
utilisateur_id	Entier	FK → UTILISATEUR	Client
recompense_id	Entier	FK → RECOMPENSE	Récompense choisie

3. Modèle Conceptuel de Données (MCD)

Le MCD ci-dessous représente les 7 entités métier reliées par leurs associations. Les cardinalités indiquent combien d'occurrences de chaque entité peuvent être liées à l'autre.



Récapitulatif des associations

Association	Entité A	Card.	Entité B	Card.
effectue	Utilisateur	1,1	Réservation	0,N
concerne	Réservation	1,1	Table	0,N
sur	Réservation	1,1	Créneau	0,N
génère	Réservation	0,1	HistoriquePoints	1,1
cumule	Utilisateur	1,1	HistoriquePoints	0,N
régit	ProgrammeFidélité	1,1	Récompense	1,N

échange	Utilisateur	0,N	Récompense	0,N
----------------	-------------	------------	-------------------	------------

L'association échange (0,N / 0,N) est de type N,N : elle se traduit en MLD par une table de liaison ECHANGE_RECOMPENSE avec ses propres attributs (date_echange, statut).

4. Modèle Logique de Données (MLD)

Chaque entité du MCD devient une table. Les associations (1,N) se traduisent par une clé étrangère côté N. L'association N,N (échange) génère la table ECHANGE_RECOMPENSE.

UTILISATEUR PK: id nom, prenom, email*, mot_de_passe, role, created_at	TABLE PK: id numero*, capacite, disponible
CRENEAU PK: id heure_debut, heure_fin, actif	RESERVATION PK: id date_reservation, nb_couverts, statut, honoree FK: utilisateur_id, table_id, creneau_id
PROGRAMME_FIDELITE PK: id points_par_resa, seuil_recompense, actif	RECOMPENSE PK: id libelle, type, seuil_points FK: programme_id
HISTORIQUE_POINTS PK: id points_gagnes, type_operation, date_operation, solde_apres FK: utilisateur_id, reservation_id?	ECHANGE_RECOMPENSE PK: id date_echange, statut FK: utilisateur_id, recompense_id

Légende : PK = clé primaire · FK = clé étrangère · * = UNIQUE · ? = nullable

5. Modèle Physique de Données (MPD)

Le script SQL ci-dessous crée le schéma complet sur MariaDB. J'ai utilisé le moteur InnoDB pour bénéficier des contraintes de clés étrangères et des transactions. Les valeurs ENUM remplacent les chaînes libres pour garantir l'intégrité des données métier.

```
-- MariaDB - Création des tables
CREATE TABLE utilisateur (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  prenom VARCHAR(50) NOT NULL,
  email VARCHAR(180) NOT NULL UNIQUE,
  mot_de_passe VARCHAR(255) NOT NULL,
  role ENUM('client','admin') NOT NULL DEFAULT 'client',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE TABLE table_resto (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  numero TINYINT UNSIGNED NOT NULL UNIQUE,
  capacite TINYINT UNSIGNED NOT NULL,
  disponible TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE TABLE creneau (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  heure_debut TIME NOT NULL,
  heure_fin TIME NOT NULL,
  actif TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE TABLE reservation (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  date_reservation DATE NOT NULL,
  nb_couverts TINYINT UNSIGNED NOT NULL,
  statut ENUM('en_attente','confirmee','annulee')
  NOT NULL DEFAULT 'en_attente',
  honoree TINYINT(1) NOT NULL DEFAULT 0,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  utilisateur_id INT UNSIGNED NOT NULL,
  table_id INT UNSIGNED NOT NULL,
  creneau_id INT UNSIGNED NOT NULL,
  FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id) ON DELETE CASCADE,
  FOREIGN KEY (table_id) REFERENCES table_resto(id),
  FOREIGN KEY (creneau_id) REFERENCES creneau(id),
  INDEX idx_date (date_reservation)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE TABLE programme_fidelite (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  points_par_resa INT UNSIGNED NOT NULL DEFAULT 10,
  seuil_recompense INT UNSIGNED NOT NULL DEFAULT 100,
  actif TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE TABLE recompense (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  libelle VARCHAR(100) NOT NULL,
  type ENUM('reduction','boisson','dessert','autre') NOT NULL,
  seuil_points INT UNSIGNED NOT NULL,
  programme_id INT UNSIGNED NOT NULL,
  FOREIGN KEY (programme_id) REFERENCES programme_fidelite(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE TABLE historique_points (
```

```
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
points_gagnes INT NOT NULL,
type_operation ENUM('gain_reservation','echange_recompense') NOT NULL,
date_operation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
solde_apres INT UNSIGNED NOT NULL,
utilisateur_id INT UNSIGNED NOT NULL,
reservation_id INT UNSIGNED DEFAULT NULL,
FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id) ON DELETE CASCADE,
FOREIGN KEY (reservation_id) REFERENCES reservation(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE echange_recompense (
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
date_echange DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
statut ENUM('utilise','annule') NOT NULL DEFAULT 'utilise',
utilisateur_id INT UNSIGNED NOT NULL,
recompense_id INT UNSIGNED NOT NULL,
FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id) ON DELETE CASCADE,
FOREIGN KEY (recompense_id) REFERENCES recompense(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6. Justifications des choix de modélisation

Normalisation

Le schéma respecte la 3NF. Chaque attribut non-clé dépend uniquement de la clé primaire de sa table, sans dépendance transitive. Par exemple, les informations du programme de fidélité (points, seuil) sont isolées dans PROGRAMME_FIDELITE et non répétées dans chaque récompense.

Choix spécifiques

- **TABLE renommée TABLE_RESTO** pour éviter le conflit avec le mot réservé SQL. Le champ *disponible* permet de désactiver une table sans la supprimer.
- **CRENEAU séparé de RESERVATION** : les créneaux sont définis une seule fois par l'admin et réutilisés à chaque réservation. Cela évite la redondance et permet de fermer un créneau sans toucher aux réservations passées.
- **PROGRAMME_FIDELITE singleton** : une seule ligne modifiable par l'admin. Simple et suffisant pour le périmètre actuel.
- **solde_apres dans HISTORIQUE_POINTS** : stocker le solde après chaque opération permet de rejouer l'historique complet sans recalcul.
- **reservation_id nullable** dans HISTORIQUE_POINTS : les échanges de récompenses génèrent aussi une ligne d'historique (points négatifs), sans réservation associée.

Intégrité référentielle

- **ON DELETE CASCADE** sur les FK vers utilisateur : suppression RGPD — toutes les données du client sont effacées avec son compte.
- **ON DELETE SET NULL** sur reservation_id dans historique : l'historique des points est conservé même si une réservation est supprimée.
- Moteur InnoDB sur toutes les tables pour le support natif des clés étrangères et des transactions.